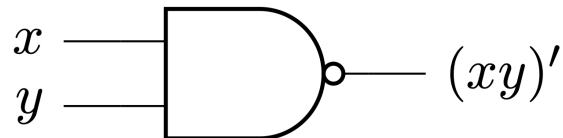


So far we have built circuits using three kinds of gates: AND, OR, and NOT. Real computer chips usually don't use those. They are built almost entirely out of NAND gates (or NOR gates), because those are cheaper and faster to make.

NAND: Not AND, an inverted AND gate. So wherever a normal AND would say 1, NAND says 0, and where AND says 0, NAND says 1.

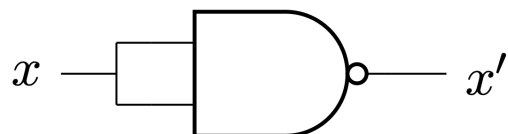
x	y	$x \cdot y$ (AND)	$(x \cdot y)'$ (NAND)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0



NAND is referred to as a **universal** gate. It is a gate that can build every other gate all by itself. We need to show how NAND can be used to build our three main operators: AND, OR, NOT.

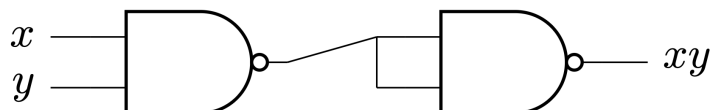
1. NAND can act as NOT:

The rule is take an input, x , and branch it off into two different inputs entering NAND. The gate computes $(x \cdot x)'$. But $x \cdot x = x$ (anything ANDed with itself is itself), so the output is x'



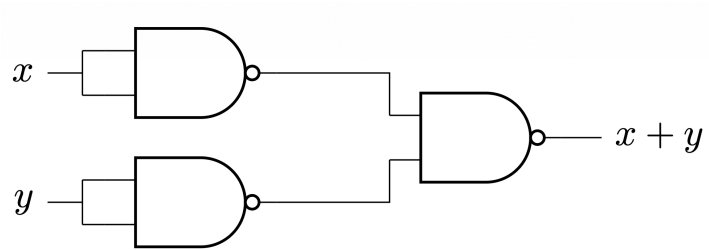
2. NAND can act as AND:

Send x and y into a NAND, then send that result through a second NAND wired.



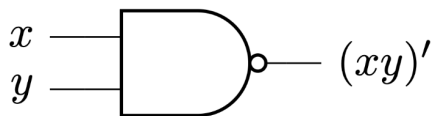
3. NAND can act as OR

Flip each input first (with NAND-inverters), then feed them into one more NAND.

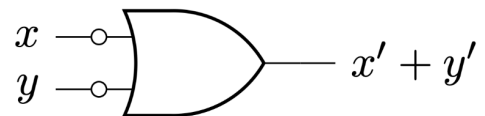


Because the method of converting our 3 operators into NAND is too complicated, there is a trick we can follow to simplify expressions. A single NAND gate can be drawn two different ways. Both drawings mean the exact same gate.

1. **Face 1 (AND-invert):** an AND shape with a bubble on the output.
2. **Face 2 (invert-OR):** an OR shape with a bubble on each input.



Face 1: AND-invert



Face 2: invert-OR

How to simplify any circuit into its NAND equivalent:

1. Simplify your function into SOP form (AND terms ORed together).
2. Every AND term uses an AND-invert
3. Every OR term uses an invert-OR
4. If there are any single literal terms, branch it off into two different inputs into an AND-invert

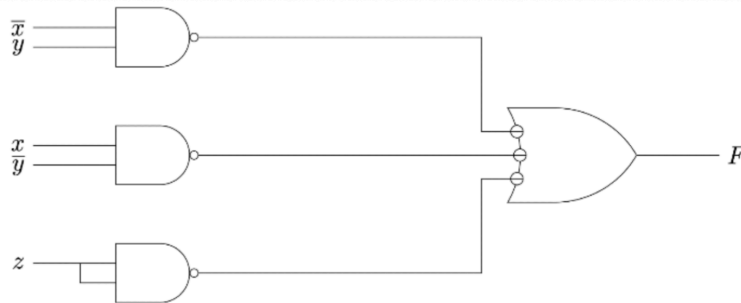
Ex) Simplify $F(x, y, z) = (1, 2, 3, 4, 5, 7)$ into its NAND circuit simplification:

Step 1 — simplify with a K-map. Put a 1 in the listed cells, then group the 1s the normal way.

		yz			
		00	01	11	10
$x = 0$		0	1	1	1
$x = 1$		1	1	1	0

The three groups give the simplified SOP:

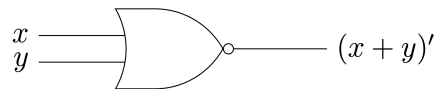
$$F = \bar{x}y + x\bar{y} + z.$$



The NOR gate follows a similar idea.

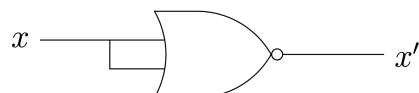
NOR: Just an OR gate with the answer flipped (“Not OR”). For NOR, you use a POS, product of sums. There are two faces, and they follow similar rules to the NAND circuit simplification.

x	y	$x + y$ (OR)	$(x + y)'$ (NOR)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0



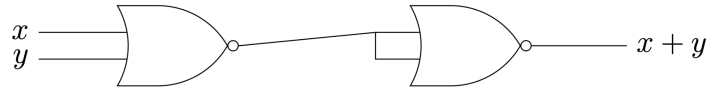
1. NOR can act as NOT:

A one-input NOR is an inverter.



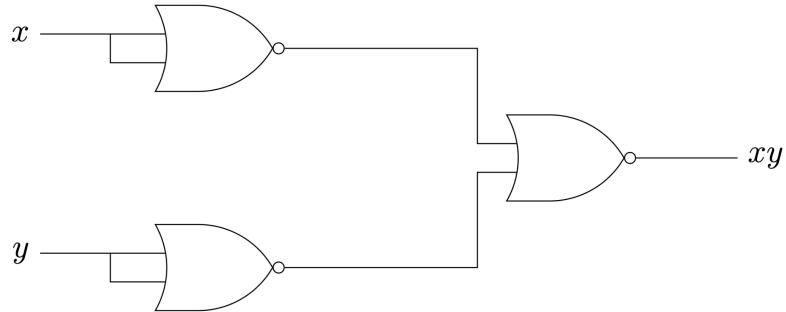
2. NOR can act as OR:

A NOR followed by an inverter.



3. NOR can act as AND:

Invert both inputs, then NOR



Ex) Simplify $F(x, y, z) = (0, 2, 4, 5, 6)$ into its NOR circuit simplification.

Step 1 — get POS by grouping the 0s. The 0s sit at minterms 1, 3, and 7.

		yz			
		00	01	11	10
$x = 0$		1	0	0	1
$x = 1$		1	1	0	1

Reading the 0-groups gives the complement $\overline{F} = \overline{x}z + yz$. Flip it with DeMorgan to get POS form:

$$F = (\overline{x}z + yz)' = (x + \overline{z})(\overline{y} + \overline{z}).$$

